DATA PREPARATION

Semester Genap 2025/2026

Pendahuluan

Data preparation merupakan proses mengubah data mentah (*raw data*) menjadi data yang bersih, terstruktur, dan siap digunakan untuk analisis atau pemodelan dalam *data mining* maupun *machine learning.* Data yang diperoleh dari berbagai sumber sering kali memiliki masalah seperti data tidak lengkap, format tidak konsisten, duplikasi, serta adanya noise atau outlier, sehingga perlu dilakukan proses pembersihan dan penyesuaian terlebih dahulu. Tahap ini merupakan bagian penting dalam metodologi **CRISP-DM** dan biasanya memakan waktu paling besar dalam proyek analisis data. Dengan demikian, *data preparation* berperan penting dalam menjaga kualitas data. Data yang berkualitas akan menghasilkan informasi yang berkualitas pula.

Tujuan dari *data preparation*:

1. Meningkatkan kualitas data
2. Menghilangkan kesalahan dan inkonsistensi data
3. Menyesuaikan format data agar dapat diproses oleh algoritma
4. Mengurangi kompleksitas data
5. Menghasilkan dataset yang optimal untuk pemodelan

Tahapan umum dalam *data preparation*:

1. *Data cleaning*
2. *Data transformation* & *normalization*
3. *Data integration*
4. *Data reduction*
5. *Feature selection*

Data Cleaning

Data cleaning adalah proses mendeteksi dan memperbaiki / membersihkan data yang tidak akurat, tidak lengkap, tidak konsisten, atau tidak relevan dalam dataset. Data yang kotor dapat menyebabkan kesalahan dalam analisis dan menurunkan performa model *machine learning*. Beberapa permasalahan yang sering ditemukan pada dataset antara lain:

1. *Missing Value*

Ini menunjukkan nilai data yang tidak tersedia atau kosong, biasanya disimbolkan dengan NA atau NaN, yaitu singkatan dari *not available* atau *not applicable*.

Sebagai contoh:

Tabel 1. Data Penderita Tuberculosis (TB)

Diagnosis	Usia	Jenis Kelamin	Merokok
TB	22	L	
TB	21	P	Tidak
TB + Penyerta	30	M	Ya
TB + Penyerta			
TB	12	L	Tidak
TB + Penyerta	32	L	Ya
TB + Penyerta		F	Ya
TB + Penyerta	56	L	
TB	58	P	Ya
TB + Penyerta		P	Ya

Pada gambar tabel di atas, semua variabel dari Usia hingga Merokok memiliki data NA, jika data ini digunakan dalam *data mining* perlu diatasi terlebih dahulu.

2. *Data Duplication*

Menunjukkan terdapat subjek/ objek yang tercatat lebih dari satu kali. Duplikasi ini apabila terjadi sangat banyak akan menyebabkan bias dalam analisis.

3. *Noise*

Noise direpresentasikan untuk data yang mengandung kesalahan atau variasi acak yang tidak relevan. Misalkan pada variabel usia terdapat data diagnose, atau usia 30 dicatat 300.

4. Data Tidak Konsisten

Data yang tidak konsisten biasanya berkaitan dengan tidak standarnya penulisan format data (misalkan Laki-laki dengan L atau Pria).

5. *Outlier*

Outlier merupakan data pencilan yang nilainya jauh dari nilai-nilai yang ada dalam satu variabel. *Outlier* ini mempengaruhi statistik atau parameter seperti mean, varians, simpangan baku, koefisien variasi. Mempengaruhi pula dari hasil analisis data seperti hasil cluster dan regresi.

Dari kelima jenis permasalahan kualitas data, dapat diatasi dengan:

1. *Imputation* dengan ukuran statistics

Solusi ini bertujuan untuk mengatasi *missing value* pada *dataset* yang digunakan. *Imputation* ini dapat menggunakan ukuran statistik, seperti mean, median, atau modus. Proses yang digunakan juga bisa disesuaikan dengan karateristik data. Umumnya, apabila data yang digunakan itu akan dianalisis menggunakan metode klasifikasi, maka data dibagi menjadi beberapa bagian yang disesuaikan dengan kelompok pada klasifikasi. Sebagai contoh pada Tabel 1 di atas, terdapat data kosong. Misalkan ingin mengatasi *missing value* pada Merokok. Karena data yang ada adalah berupa kategorikal, maka yang cocok menangani *missing value* adalah menggunakan modus. Misal, penanganannya diklasifikasikan berdasarkan diagosisnya, maka code dapat menggunakan:

```
import pandas as pd
import numpy as np
# import data
df = pd.read_csv("tb_pasien.csv")

# cek missing value
print(df.isnull().sum())

# imputasi merokok berdasarkan diagnosis
df['Merokok'] = df.groupby('Diagnosis')['Merokok'] \
                    .transform(lambda x: x.fillna(x.mode()[0]))
# cek hasil
print(df)

# simpan data baru
df.to_csv("tb_pasien_clean.csv", index=False)
```

Untuk variabel usia dapat menggunakan mean atau median. Untuk data pada urutan keempat dapat dihapus.

2. Penghapusan data duplikat

Untuk data duplikat, dapat mengecek dan menghapusnya dengan perintah:

```
df.duplicated()
df = df.drop_duplicates()
```

3. Standarisasi format data

Misalkan pada variabel Jenis Kelamit terdapat ketidakkonsistenan penulisan, yaitu untuk laki-laki ada yang ditulsikan L dan M atau perempuan ada yang dituliskan P dan F. Kita ingin mengubah ke dalam bentuk yang konsisten, yaitu laki-laki dengan simbol L dan perempuan dengan simbol P, maka codenya adalah:

```
df['Jenis Kelamin'] = df['Jenis Kelamin'].replace({
'M':'L',
'F':'P',
})
```

4. Penghapusan data pencilan/ *outlier*

Penghapusan data pencilan/ *outlier* bertujuan untuk membuat data lebih seragam dan tidak ada nilai ekstrim. Contoh code, di mana data masih sama seperti pada Tabel 1 dan code merupakan lanjutan dari code imputation:

```
# Menghitung Q1 dan Q3
Q1 = df['Usia'].quantile(0.25)
Q3 = df['Usia'].quantile(0.75)

# Menghitung IQR
IQR = Q3 - Q1

# Menentukan batas bawah dan atas
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Menampilkan outlier
outliers = df[(df['Usia'] < lower_bound) |
(df['Usia'] > upper_bound)]
print("Data Outlier:")
print(outliers)

# Menghapus outlier
df_clean = df[(df['Usia'] >= lower_bound) &
(df['Usia'] <= upper_bound)]

print("Data setelah outlier dihapus:")
print(df_clean)
```

Untuk data *noise* cara penangannya secara umum sama seperti penanganan *outlier* ketika data yang tertulis adalah salah input, yaitu deteksi *outlier* terlebih dahulu. Kemudian ada penggantian data *noise* dengan menggunakan median atau rata-rata. Contohnya, usia pasien harusnya 30 tetapi tertulis 300. 300 ini telah terdeteksi sebagai *outlier* karena berada di luar *lower* ataupun *upper bound*. Angka 300 itu misalkan diganti dengan angka 32, di mana 32 ini misalkan berasal dari rata-rata usia dari semua pasien. Contoh coding adalah:

```
# 1. Menghitung Q1 dan Q3
Q1 = df['Usia'].quantile(0.25)
Q3 = df['Usia'].quantile(0.75)

# 2. Menghitung IQR
IQR = Q3 - Q1

# 3. Menentukan batas bawah dan atas
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# 4. Menampilkan outlier
outliers = df[(df['Usia'] < lower_bound) | (df['Usia'] > upper_bound)]
print("Data Outlier:")
print(outliers)

# 5. Hitung rata-rata usia dari data yang valid (tidak termasuk outlier)
mean_valid = df[(df['Usia'] >= lower_bound) & (df['Usia'] <= upper_bound)]['Usia'].mean()

# Ganti nilai outlier dengan rata-rata valid
df.loc[(df['Usia'] < lower_bound) | (df['Usia'] > upper_bound), 'Usia'] = mean_valid

print("\n=== Data Setelah Penggantian Noise ===")
print(df)
```

Data Transformation & Standarization

Transformasi data adalah proses mengubah format atau skala data agar lebih sesuai untuk analisis atau pemodelan. Transformasi sangat penting terutama pada algoritma *machine learning* yang sensitif terhadap skala data. Sedangkan normalisasi/ standarisasi adalah bagian dari transformasi data. Normalisasi/ standarisasi berfungsi untuk mengubah skala data dengan mean = 0, dan variance = 1. Selain standarisasi, terdapat beberapa macam jenis transformasi data, yaitu *encoding* data kategori (mengubah data ketegori menjadi numeric, seperti L = 1, P = 2), *discretization* (mengubah data numeric menjadi data kategori atau interval, seperti IPK dibuat memuaskan, sangat memuaskan, cumlaude, dan summa cumlaude), dan *log transformation* (mengubah data menggunakan fungsi log untuk mengurangi skewness pada data). Contoh code:

```
#encoding Jenis Kelamin
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Jenis_Kelamin_encode'] = le.fit_transform(df['Jenis Kelamin'])
print(df[['Jenis Kelamin','Jenis_Kelamin_encode']])

#Bining Usia menjadi kelompok Usia
df['Kelompok_Usia'] = pd.cut(df['Usia'],
bins=[0,14,24,44,100],
labels=['Anak','Remaja','Dewasa','Lansia'])
print(df[['Usia','Kelompok_Usia']])

#Standarisasi dengan Z score
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['Usia']] = scaler.fit_transform(df[['Usia']])
```

Data Integration

Data Integration adalah proses **menggabungkan data dari berbagai sumber yang berbeda menjadi satu dataset yang konsisten dan terpadu**.

Dalam praktik analisis data, informasi sering berasal dari berbagai sumber seperti:

1. database rumah sakit
2. file CSV
3. spreadsheet
4. sistem informasi yang berbeda

Karena berasal dari sumber yang berbeda, data tersebut sering memiliki:

- struktur berbeda
- format berbeda
- penamaan variabel berbeda
- kemungkinan duplikasi data

Oleh karena itu diperlukan proses **integrasi data** sebelum dilakukan analisis lebih lanjut.

Misalkan dari Tabel 1 itu berasal dari 2 dataset, yaitu dataset 1 terdiri dari ID, Diagnosis, dan Usia. Sementara untuk dataset kedua terdiri dari ID, Jenis Kelamin, dan Merokok. Dari kedua dataset ini akan digabung menjadi satu dataset, maka contoh codenya adalah:

```
import pandas as pd
#import data
pasien = pd.read_csv("data_pasien.csv")
kebiasaan = pd.read_csv("data_kebiasaan.csv")

#menggabung dataset
data = pd.merge(pasien, kebiasaan, on="ID")
print(data)
```

Data Reduction

Salah satu contoh dari *data reduction* adalah *dimensionality reduction*. Dimensionality reduction adalah proses mengurangi jumlah variabel dalam dataset tanpa kehilangan terlalu banyak informasi. Dataset dengan banyak variabel dapat menyebabkan:

- overfitting
- waktu komputasi tinggi
- model kompleks

Salah satu contoh dari *dimensionality reduction* adalah menggunakan metode *Principal Component Analysis* (PCA). Contoh code:

```
#dari data integration

from sklearn.decomposition import PCA
pca = PCA(n_components=2)
principal_components = pca.fit_transform(data[['Jenis Kelamin','Merokok']])

print(principal_components)
```

Feature Selection

Feature selection adalah proses memilih variabel (fitur) yang paling relevan dari suatu dataset untuk digunakan dalam analisis atau pemodelan machine learning. Tujuannya adalah mengurangi jumlah variabel yang tidak penting, meningkatkan kinerja model, serta membuat model lebih sederhana dan mudah diinterpretasikan. Beberapa metode dalam *feature selection* adalah ***Filter Method.***

Filter method ini adalah metode pemilihan variabel berdasarkan hubungan keeratan antar variabel, seperti menggunakan analisis korelasi atau uji Chi-square.
Contoh code, misalkan dari data pada Tabel 1 diberikan tamban variabel Suhu Tubuh, Berat Badan, dan Batuk 2 Pekan (Ya, Tidak):

```
import pandas as pd
data = pd.read_csv("tb_pasien.csv ")

# memilih variabel numerik
numeric_data = data[['Umur','Suhu_Tubuh','Berat_Badan','TB']]

# menghitung korelasi
correlation = numeric_data.corr()
print(correlation['TB'].sort_values(ascending=False))

#Chi-square
data['Batuk 2 Minggu'] = data['Batuk 2 Minggu'].map({'Tidak':0,'Ya':1})
data['Merokok'] = data['Merokok'].map({'Tidak':0,'Ya':1})

X = data[['Batuk 2 Minggu','Merokok']]
y = data['TB']

chi = SelectKBest(score_func=chi2, k='all')
chi.fit(X,y)

print(chi.scores_)
```

Referensi

Han, Jiaei, Jian Pei, and Hanghang Tong. 2023. *Data mining* Concepts and Techniques. 4th ed. Beth LoGiudice. Cambridge: Katey Birtcher.